

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 9

Stacks in Action

Course Logistics

- Quiz 1 (CS213 and CS293) dates announced
 - CS293: Tuesday Aug 25, during lab hours
 - CS213: Thursday Aug 27, lecture hour (rooms TBA)
- Partial-credits policy for Weekly-Lab Graded Submission announced
 - You have only two opportunities for claiming partial credits. Use wisely.
- CS213 Participation points: starting next week
 - Bring your device (laptop/phone)

Stack

Recall

- What is a stack?
 - Objects (say books) are kept one on top of the other
 - An item can be added only to the top of the stack
 - Can remove only the top-most item (the most recently added)
 - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
 - To support "undo", a stack can be used to store actions so far
 - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
 - Systematic searches often use a stack too
 - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
 - ...



Today: Some More Examples

- Implicit stacks in recursive functions
- Balanced Brackets
- A Reverse Polish Notation (RPN) calculator
- An application of monotonic stack

Implicit Stack in Recursion

- When a function is called, memory for its local variables are allocated on a stack
- When the function returns, the stack is popped
- Consider calling the following function with input 7890

```
1. void printdigits(unsigned int n) {  
2.   if(n>=10) { printdigits(n/10); }  
3.   std::cout << " " << n%10;  
4. }
```

console

7 8 9 0

n: 7 pc:4

n: 78 pc:4

n: 789 pc:4

n: 7890 pc:4

Implicit Stack in Recursion

```
void printdigits_stack(unsigned int n) {  
    std::stack<unsigned int> s; s.push(n);  
    while(n>=10) { s.push(n/=10); }  
    while (!s.empty()) { std::cout << " " << s.top()%10; s.pop(); }  
}
```

```
1. void printdigits(unsigned int n) {  
2.     if(n>=10) { printdigits(n/10); }  
3.     std::cout << " " << n%10;  
4. }
```

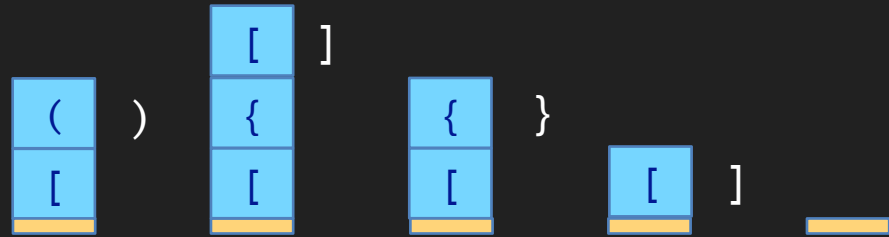
Balanced Brackets

- Consider a string with three kinds of brackets: `()`, `{}` and `[]`
- In a valid string, they should be properly nested: e.g., `[ ( ] )` is not OK
 - What is valid?
 - Has the form empty-string, `[ valid ]`, `( valid )`, `{ valid }`, or `valid valid`
 - Either breaks into two shorter (non-empty) valid strings
 - Or the first character should be an opening bracket, and it should be closed/matched by the last one; after stripping off those two, the remaining should be a valid string (of shorter length)
 - How do we check if a string is valid: looking at the first/last characters can't tell which form it is: e.g., `[ ( ) { } ]` vs. `[ ] ( ) [ ]`

Balanced Brackets

- Check using a stack, scanning the string left-to-right once
- Each opening bracket is pushed into the stack
- When a closing bracket encountered, pop the stack to see if it matches
- While popping, if there is a mismatch or if the stack empty, unbalanced
- If the string ends while the stack is not empty, unbalanced
- Otherwise (the string ends with no mismatch, and the stack is empty at that point) balanced

[() { [] }]



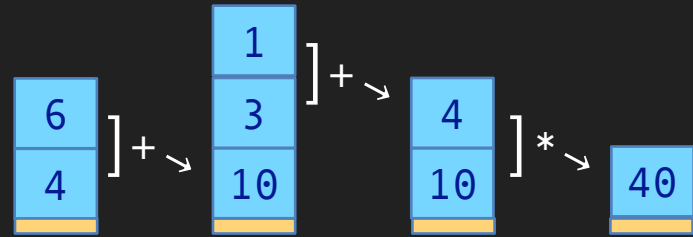
Balanced Brackets

- Claim: Exactly balanced bracket strings are accepted as balanced by this program
- Proof: By induction on the length of the string.
 - Base case: length 0 string. It is balanced, and our program accepts it as balanced ✓
 - Induction: For any $k > 0$, suppose it holds for all strings of length $< k$. For a k -long string:
 - If valid, will accept: If of the form $A B$, where each is valid, non-empty, then by induction hypothesis, our stack will be empty after scanning A successfully, and then will be empty again after scanning B successfully. ✓ If of the form $\{A\}$, first we push $\{$ and then run the program on A , which by induction hypothesis, will return it to $\{$, having never popped it; then on seeing $\}$, will pop it. ✓
 - If accepts, then valid: If we accept, first character is one of $[$, $($ or $\{$. Two cases: stack ever became empty before the end, or not. If it became empty after scanning a prefix A of a string AB , then it again became empty after scanning the suffix B ; ie it accepts A and B each, and by induction hypothesis, both are valid. Then, by definition, AB is also valid. ✓ If it never became empty till the end, the first opening bracket, say $\{$, stayed at the bottom of the stack throughout and was popped at the end by a matching closing bracket. So the string is of the form $\{A\}$; further, if run on A , the program will accept (all conditions met — check!). So A is valid by induction hypothesis. ✓

RPN Calculator

- Standard mathematical expressions need brackets
 - $(a+b) * (c+d)$
- In Reverse Polish Notation (RPN), no brackets needed
- In RPN, also known as postfix notation, the operator is placed after the operands (arg1 arg2 op instead of arg1 op arg2)
 - Instead of $(a+b) * (c+d)$ we have $a \ b \ + \ c \ d \ + \ *$
- Makes it very easy to evaluate an expression in an input stream, using a stack of numbers

4 6 + 3 1 + *

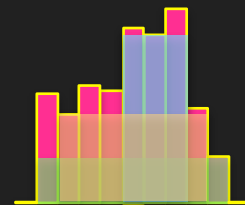


RPN Calculator

```
enum token_t { plus, minus, star, slash, num };
struct token { token_t ty; double val; };
bool rpnCalc(const std::vector<token>& tokens, double& result) {
    std::stack<double> s;
    for(const token& t: tokens) { // iterate through all the tokens
        if (t.ty == num) { s.push(t.val); } // numbers go into the stack
        else { // apply operator on top two items on stack
            if (s.size() < 2) return false; // stack underflow!
            double b = s.top(); s.pop();
            double a = s.top(); s.pop();
            switch(t.ty) {
                case plus: s.push(a + b); break;
                case minus: s.push(a - b); break;
                case star: s.push(a * b); break;
                case slash: if (b == 0) return false; s.push(a / b); break; //divide by 0 is error
                default: return false; // won't get here (unless the code missed a token_t)
            }
        }
    }
    if (s.size() != 1) return false; // correct expressions leave a single value on stack
    result = s.top(); return true;
}
```

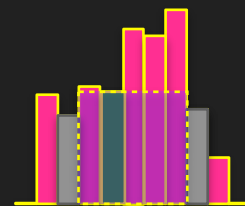
Largest Rectangle in a Patchwork of Strips

- Equal-width but varying-length strips of cloth are sewn together, with one end aligned, to form a large patchwork
 - What is the largest area of a rectangle aligned with the straight edge, that can be cut-out of this?
- Call a rectangle *maximal* if there is a “limiting strip” whose height (i.e., length) equals the rectangle’s height, and the strips to the rectangle’s immediate left and right are strictly shorter
- The largest rectangle is maximal, imagining sentinel strips of height 0 to either end
- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
 - Then the max among those is the largest rectangle



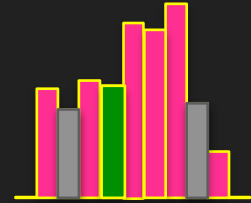
Largest Rectangle in a Patchwork of Strips

- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip
- Naively: scan the strips to the left and to the right, starting from this strip. $\Theta(n)$ time.
 - To do this for all strips, we need $O(n^2)$ time. In fact, $\Theta(n^2)$ time (eg. if all the strips are the same length).
- Goal: $O(n)$ total time



Largest Rectangle in a Patchwork of Strips

- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip
- Scanning left to right, push each strip into a stack such that heights are strictly monotonically increasing
 - When a strip enters the stack we pop entries taller than it. So its closest shorter strip to the left is just under it in the stack
 - When that strip is later popped, it is popped by its closest shorter strip to the right (the ones that didn't pop it are taller)
 - At that point, we know both the neighbours we were looking for



Each item is pushed in once and popped out once

